Звіт до програмного проєкту 4, варіант 18

1. Виконавець: Старченко Ярослав Сергійович, група К-28.

2. Структура даних для багатопотокової програми з m = 2 цілих полів, кожне з яких можна окремо читати та змінювати, з перевантаженим `operator string`. Частки операцій: поле 0 — read 20%, write 10%; поле 1 — read 25%, write 10%; string 35%.

3. Використано два м'ютекси `std::shared_mutex`, по одному на поле. Кожен захищає лише своє поле. `read i` бере спільне блокування (`shared_lock`) м'ютекса поля i, `write i` — виключне (`unique_lock`) м'ютекса поля i. `string` бере спільне блокування обох м'ютексів, завжди в порядку 0, 1, копіює обидва значення й форматує рядок уже після звільнення м'ютексів. Обґрунтування: 80% операцій лише читають (20 + 25 + 35), тому спільні блокування дають змогу виконувати їх одночасно, а записи становлять тільки 20%. Окремий м'ютекс на поле означає, що запис у поле 0 не зупиняє роботу з полем 1 і навпаки. `string` блокує обидва поля, щоб отримати узгоджений знімок, а найдорожчу частину (форматування) виконує поза блокуванням. Кожне поле разом із м'ютексом вирівняне на 64 байти, щоб уникнути false sharing. Взаємоблокування неможливе: два м'ютекси одночасно бере тільки `string`, і завжди в однаковому порядку. Атомарні змінні не використовуються.

4. Генератор створив файли з 300 000 операцій для кожної кількості потоків (по 300 000 / k у файлі кожного з k потоків) з точними частками у випадковому порядку: a) частоти за умовою; b) усі частоти по 20%; c) 5 / 40 / 5 / 40 / 10 (read 0, write 0, read 1, write 1, string), тобто переважають записи. Усі потоки працюють з однією спільною структурою. Збірка Release (MSVC, C++20, `/O2`). Час без зчитування файлів, середнє з 10 запусків після розігріву, мс:

файл	**1 потік**	**2 потоки**	**3 потоки**
a) за умовою	23.450	26.465	34.525
b) рівні частоти	17.485	20.656	27.969
c) переважають записи	10.387	23.986	26.517

5. Результати відповідають очікуванням. Кожна операція дуже коротка, тому основні витрати в кількох потоках — змагання за кеш-лінії м'ютексів, і прискорення немає ні в одному файлі. Проте для файлу a), під який оптимізовано структуру, перехід з 1 до 2 потоків збільшує час лише на 13%, а для b) — на 18%. Для c), де переважають виключні блокування, той самий перехід збільшує час на 131%. Отже, схема найкраще працює саме з частотами варіанту. Абсолютний час для a) найбільший, бо там найбільше операцій string (35%), а форматування рядка — найдорожча операція.

6. Реалізував клас `Fields` (`include/fields.h`): поля зберігаються у звичайному масиві з двох структур `Slot`, кожна містить `int` і власний `std::shared_mutex` та вирівняна через `alignas(64)`; методи `get`/`set` з перевіркою номера поля і `explicit operator std::string`, що копіює значення під `shared_lock` і форматує через `std::format`. Написав генератор файлів (`src/generator.cpp`): точна кількість операцій кожного виду, перемішування через `std::shuffle` з `std::mt19937` і фіксованим зерном, окремий файл для кожного потоку. Написав програму вимірювання (`src/main.cpp`): розбір файлів у `std::vector` операцій до початку виміру, одночасний старт потоків `std::thread` через `std::mutex` і `std::condition_variable`, вимір часу через `std::chrono::steady_clock`, розігрів і усереднення 10 запусків, контрольна сума результатів читань, щоб компілятор їх не викинув. Зібрав у Release (скрипт `build.cmd` і рішення Visual Studio), провів виміри й проаналізував результати.